Sheaf Refinement Types for Beginners

A 200-slide mini-course

No category theory or PL theory required

Halley Young

1. What **types** are and why they matter
2. What **refinement types** add on top
3. The **problem**: different parts of a program can disagree
4. The **jigsaw-puzzle idea** (the sheaf concept, no math)
5. How **DepPy** actually scans your code
6. Finding **bugs** automatically (20 classes, 112 types)
7. **Checking specifications**: does your code do what you intended?
8. **Comparing programs**: are two implementations equivalent?

- **No category theory.** We use analogies, not definitions.
- **No PL theory.** No lambda calculus, no Hoare logic.
- **No advanced math.** The hardest concept is "if / else."
- You *do* need: basic Python (variables, functions, `if`).

If you can read a 10-line Python function, you're ready.

What Is a Type?

A **value** is a piece of data your program works with.

`42`	a number
`"hello"`	a piece of text
`[1, 2, 3]`	a list of numbers
`True`	a yes/no answer
`3.14`	a decimal number

Every value in every program is *some* kind of thing.

A **type** is a name for a kind of value.

Value	Type	English
`42`	`int`	integer (whole number)
`"hello"`	`str`	string (text)
`[1,2,3]`	`list`	list (ordered collection)
`True`	`bool`	boolean (yes or no)
`3.14`	`float`	floating-point (decimal)

Types tell us **what operations make sense**.

Operation	Makes sense?	Why?
`42 + 10`	✓	both are `int`
`"hi" + " world"`	✓	both are `str`
`"hi" + 42`	×	can't add str and int
`len([1,2,3])`	✓	lists have length
`len(42)`	×	numbers don't

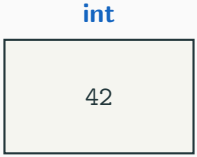
When you use a value in a way that doesn't match its type, you get a **type error**.

```
>>> "hello" + 42
TypeError: can only concatenate str to str
```

```
>>> len(42)
TypeError: object of type 'int' has no len()
```

A type error = using a screwdriver as a hammer. Wrong tool for the job.

Think of a type as a **label on a box**.



The label says what *kind* of thing is inside.
If a function expects a box labeled `int`, don't hand it one labeled `str`.

Two philosophies:

Static typing (Java, C, Rust): Check labels **before** running.
If a label is wrong, the program won't even start.

Dynamic typing (Python, JavaScript): Check labels **while running**.
If a label is wrong, the program crashes mid-way.

Python is dynamically typed. You only discover the mistake when the code actually runs that line.

Python lets you *optionally* add labels (hints):

```python
def greet(name: str) -> str:
    return "Hello, " + name
```

- `name: str` — "name should be a string"
- `-> str` — "this function returns a string"

But Python **doesn't enforce** these hints at runtime.
Tools like `mypy` check them before you run.

Types can tell you:

- "`x` is an `int`" — so you can add, subtract, compare it
- "`f` returns a `list`" — so you can index into the result

Types **can't** tell you:

- "`x` is a *positive* integer"
- "this list is *sorted*"
- "this string is *not empty*"
- "this number is *not zero*"

Ordinary types say *what kind*, but not *which values*.

Ordinary types leave a **gap** between what they guarantee and what the code actually needs.

Refinement types close this gap.

- **Type** = "You have a boarding pass" (you're a passenger)
- **Refinement** = "Your boarding pass is for *this flight*, *this seat*"

Having *a* boarding pass isn't enough.
It has to be the *right* boarding pass.

Same with code: having *an* int isn't enough.
It has to be the *right kind* of int (nonzero, positive, in range, . . .).

```python
def average(total: int, count: int) -> float:
    return total / count   # What if count is 0?
```

- mypy says: ✓ types are fine (`int / int` → `float`)
- Reality: × crashes when `count = 0`

The type system sees "two ints" and says OK.
But the *value* of `count` matters, and types don't track values.

```python
def first_word(text: str) -> str:
    return text.split()[0]  # What if text is ""?
```

- `mypy`: ✓ str in, str out
- Reality: × `IndexError` when `text = ""`

The type says "it's a string." But we need "it's a *nonempty* string."

```python
def get_element(items: list, index: int):
    return items[index]  # What if index >= len(items)?
```

- Types say: list + int → OK
- Reality: `IndexError` if index is out of bounds

We need: "`index` is between 0 and `len(items) - 1`."

All three bugs have the same shape:

1. The **type** is correct (`int`, `str`, `list`)
2. But the code needs **more** than just the type
3. It needs a **promise about the value**

Bug	Missing promise
Division by zero	`count != 0`
Empty string	`len(text) > 0`
Index out of bounds	`0 <= i < len(items)`

Types catch wrong kinds of values.

But most real bugs are wrong values of the right kind.

A way to say:

- "This is an `int` **and it's not zero**"
- "This is a `str` **and it's not empty**"
- "This is an `int` **and** $0 \leq \texttt{i} < \texttt{len(items)}$"
- "This is a `list` **and it's sorted**"

In other words: **type + promise about the value**.

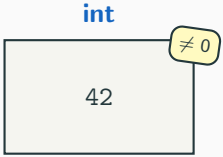
This is exactly what a *refinement type* is.

- A **value** is a piece of data
- A **type** names the *kind* of data (`int`, `str`, ...)
- Types catch some errors, but miss bugs about *which* values
- We need something stronger: **type + promise**

Next: Refinement Types — types with promises attached.

Refinement Types

Refinement type = ordinary type + a promise about the value.



The box label says "int." The sticky note adds "and it's not zero."

We'll write refinement types like this:

```
int where x != 0
```

Read it as: "an integer `x` such that `x` is not zero."

More examples:

- `str` where `len(s) > 0` — a nonempty string
- `list` where `len(L) <= 100` — a list with at most 100 elements
- `float` where `0.0 <= x <= 1.0` — a probability

When you write:

- `total / count` — you're *assuming* `count != 0`
- `items[i]` — you're *assuming* `0 <= i < len(items)`
- `result.strip()` — you're *assuming* `result is not None`
- `sorted_list[0]` — you're *assuming* the list isn't empty

Every indexing, every division, every method call has a **hidden promise**.
You just never write it down.

- The **box** = the variable and its type
- The **sticky note** = extra promise (refinement)
- Together = a refinement type

The number 42 satisfies *many* refinements at once:

- `x != 0` ✓
- `x > 0` ✓
- `x < 100` ✓
- `x % 2 == 0` ✓ (it's even)
- `x == 42` ✓ (the most precise)

Refinements form a **spectrum** from vague ("it's an int") to precise ("it's exactly 42").

Some promises *imply* others:

$$\texttt{x > 0} \xrightarrow{\text{implies}} \texttt{x != 0} \xrightarrow{\text{implies}} \texttt{x is int}$$

If `x > 0`, then automatically `x != 0`.

A **stronger** promise gives you more guarantees.

A **weaker** promise is easier to satisfy.

Refinements work on *any* type:

Type	Refinement	Meaning
`int`	`x > 0`	positive integer
`str`	`len(s) > 0`	nonempty string
`list`	`sorted(L) == L`	sorted list
`dict`	`"name" in d`	dict with a “name” key
`float`	`not math.isnan(x)`	not-NaN float
`file`	`f.closed == False`	open file handle

You can combine multiple promises:

`int` where `x > 0` **and** `x < 100`

"A positive integer less than 100."

`list` where `len(L) > 0` **and** `sorted(L) == L`

"A nonempty, sorted list."

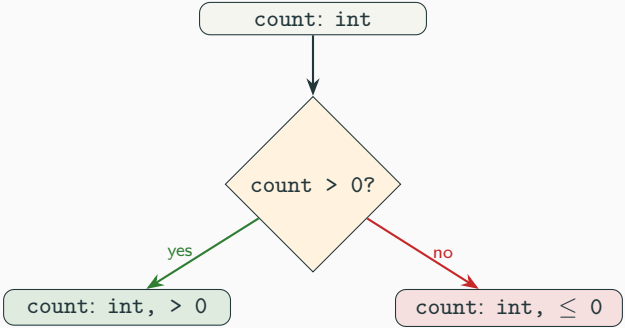
More promises = more precise = catches more bugs.

Refinements arise naturally from your code:

```
if count > 0:           # after this line:
    avg = total / count # count has refinement: > 0
```

```
if result is not None:  # after this line:
    print(result.upper())  # result: not None
```

Every `if`-statement **creates** a refinement inside its branch.



An `if`-test **splits** a type into two refined versions.

What a function *needs* from its inputs is a refinement:

```
def binary_search(arr, target):
    # NEEDS: arr is sorted
    lo, hi = 0, len(arr) - 1
    ...
```

The function works correctly *only if* `arr` is sorted.

That's a refinement on the input: `list` where `sorted(arr) == arr`.

What a function *guarantees* about its output is also a refinement:

```
def abs_val(x):
    if x < 0:
        return -x
    return x
    # GUARANTEES: result >= 0
```

The output has refinement: `int` where `result >= 0`.

	Input refinement	**Output refinement**
Who writes it?	The caller must provide it	The function must ensure it
Who benefits?	The function uses it	The caller uses it
Analogy	"Bring your passport"	"Here is your visa"

A function is a **contract**: I need *this*, and in return I give you *that*.

When functions call each other, promises chain:

```
def get_count():    # returns: int (might be 0)
    return len(data)

def average():
    c = get_count() # c: int (might be 0)
    return total / c  # NEEDS: c != 0   <-- PROBLEM!
```

`get_count` promises "an int."

`average` needs "an int that isn't zero."

The promise isn't strong enough.

Refinements **flow** from one place to the next.

When the flow breaks — when what's provided doesn't meet what's needed — that's a bug.

At every point where data flows from A to B:

Available (what A provides)	`int`
Required (what B needs)	`int` where `x != 0`

Bug = Required is *stronger* than Available.
Safe = Available is *at least as strong as* Required.

```python
def average():
    c = get_count()   # c: int (might be 0)
    if c == 0:
        return 0.0    # early exit: safe
    return total / c  # NOW c: int, != 0  -- safe!
```

The `if` guard **creates** the missing refinement.

After the guard, we *know* `c != 0`, so division is safe.

Tests check *specific* inputs:

- `test_average(10, 2)` → 5.0 ✓
- `test_average(0, 0)` → crash ×

But you have to *think of* the bad input.
If you forgot to test `count = 0`, the test suite passes and the bug hides.

Refinement types check *all possible* inputs by reasoning about promises, not examples.

> Tests: "these 5 inputs work." Refinements: "*every* input works (or here's one that doesn't)."

Type = what kind of box

Refinement = a sticky note on the box

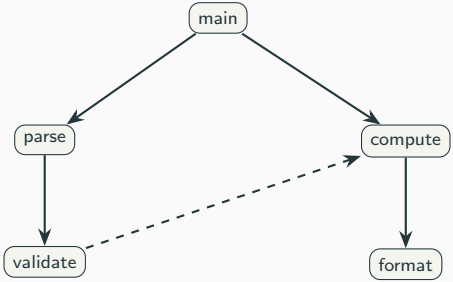
Bug = a sticky note that's missing
where the next function needs one

- A **refinement type** = type + promise about the value
- Refinements arise from `if`-guards, function contracts, computations
- They flow through the program along data paths
- A bug = a place where the **available** refinement is weaker than the **required** one
- Tests check examples; refinements check *all* values

Next: What happens when promises break?

The Problem: Promises Break

A real program isn't one function — it's **hundreds** of functions calling each other.



Each arrow is a place where data flows — and promises must match.

Function	Promises about input	Promises about output
`parse`	takes a string	returns a dict
`validate`	takes a dict	returns a dict with "name" key
`compute`	takes a dict with "name"	returns an int > 0
`format`	takes an int > 0	returns a string

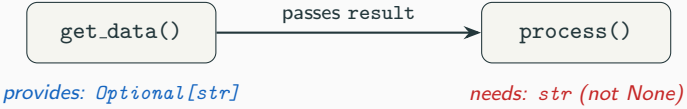
Each function has its own input and output promises (refinements).

When function A calls function B:

A's **output promise** must be $\geq$ B's **input requirement**

If A promises "an int" but B needs "an int > 0":

- A might hand over 0 or −5
- B will crash or produce wrong results
- The **handoff** is the problem



`get_data()` might return `None`.
`process()` calls `.strip()` on it — crash!

The arrow between the two functions is where the bug lives.

```python
def get_count(data):
    return len(data)    # might be 0

def average(data, total):
    count = get_count(data)
    return total / count  # BOOM if count == 0
```

Available: `int` (could be anything)

Required: `int` where `x != 0`

Verdict: Promise too weak

```python
def find_user(db, name):
    return db.get(name)  # might be None

def greet(db, name):
    user = find_user(db, name)
    print("Hi " + user.display_name)  # BOOM if None
```

Available: `Optional[User]` (might be None)

Required: `User` (not None)

Verdict: Promise too weak

```python
def get_first(items):
    return items[0]     # BOOM if items is empty

def process(data):
    if data:
        items = parse(data)
    else:
        items = []
    return get_first(items)  # items might be []
```

Available: `list` (might be empty)

Required: `list` where `len(L) > 0`

```python
def get_setting(config, key):
    return config[key]  # BOOM if key not in config

settings = {"host": "localhost"}
port = get_setting(settings, "port")  # KeyError!
```

Available: `dict` (might not have that key)

Required: `dict` where `key in d`

```python
def build_message(name, age):
    return "Hello " + name + " age " + age
    # BOOM if age is int, not str
```

Available:	`int` (from caller)
Required:	`str` (for concatenation)
Verdict:	Carrier mismatch — entirely wrong type

Every example has the same shape:

1. Function A produces a value with promise P
2. Function B needs a value with promise Q
3. P does not imply Q
4. **Bug.**

$P \not\Rightarrow Q$ at a connection $=$ a bug.
$P \Rightarrow Q$ at every connection $=$ no bugs.

Promise mismatches happen not just between functions, but:

- Between **branches**: the `if`-branch sets a variable, the `else`-branch doesn't
- Between **loop iterations**: a value changes inside the loop
- Between **modules**: one file exports a value, another imports it
- Between **threads**: two threads access the same data

Anywhere data flows from one place to another, promises must match.

```python
def process(flag):
    if flag:
        x = 42
    # else: x is never assigned!
    return x   # BOOM if flag is False
```

One branch promises "x exists." The other branch makes no such promise.

After the `if`, we can't be sure `x` is defined.

```python
def countdown(n):
    while n > 0:
        pass  # oops, n never changes!
    return n
```

Before the loop: `n > 0`.

The loop body should make progress toward `n <= 0`.

But it doesn't — so the loop never ends.

Non-termination = a promise about "eventually finishing" that's broken.

```python
f = open("data.txt", "r")
data = f.read()
f.close()
print(f.read())  # BOOM: file is already closed!
```

After `f.close()`: the promise "f is open" becomes "f is closed."

`f.read()` needs "f is open" — mismatch!

```python
shared = []
def worker1():
    shared.append(1)  # no lock!
def worker2():
    shared.append(2)  # no lock!
```

Both threads promise "I can modify `shared`."

But neither promises "I have exclusive access."

Result: data race.

Program size	Approximate connections
10-line script	∼5
100-line module	∼50
1,000-line package	∼500
10,000-line app	∼5,000

Every connection is a place where promises must match.
Checking them all by hand is impossible.
We need **automation**.

- Promises are **implicit** — programmers rarely write them down
- They **flow** through many layers of function calls
- They **change** at every `if`-branch, loop iteration, and exception handler
- A bug might involve a chain of 5+ functions

We need a **systematic framework** for:

1. Discovering what each part promises
2. Checking that adjacent promises agree
3. Reporting exactly where they don't

Imagine a tool that could:

1. Scan your code automatically
2. Find every "observation point" (variable, function input, branch, . . .)
3. Record the promise at each point
4. Check every connection: does promise A imply promise B?
5. Report mismatches as bugs, with exact locations

This is exactly what DepPy does. And the theory behind it is the "sheaf" idea.

Local promises that fit together consistently form a global guarantee.

Local promises that don't fit together reveal exactly where the bugs are.

- Programs have many parts, each with its own promises
- Where parts connect, promises must **agree**
- Mismatches = bugs (div-by-zero, null deref, OOB, . . .)
- Mismatches happen between functions, branches, loops, threads
- Checking by hand doesn't scale — we need a framework

Next: The Jigsaw Puzzle — a visual way to think about this.

The Jigsaw Puzzle

Your program is a **jigsaw puzzle**.

Each piece = one part of your code.

Each edge = a connection between parts.

The picture on each piece = the promise it makes.

A **piece** is any chunk of code that has its own promise:

- A function
- One branch of an `if`/`else`
- A single variable at a particular point
- A loop body
- A return statement

Each piece "knows" something about the values passing through it.

An **edge** is where two pieces touch — where data flows from one to another:

- Function A calls Function B $\rightarrow$ edge at the call site
- `if`-branch flows into code after the `if` $\rightarrow$ edge at the join
- Loop iteration n flows into iteration $n + 1$ $\rightarrow$ edge at the loop header
- Module A imports from Module B $\rightarrow$ edge at the import

The "picture" on a piece is its **local promise** (refinement):

Piece A	Piece B
`count`: int, > 0	`count`: int, $\neq 0$

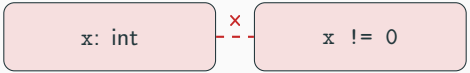
Piece A says: "count is a positive integer."
Piece B says: "count is a nonzero integer."

Two pieces **match** at their shared edge if:

Piece A's promise **implies** Piece B's requirement

`x > 0` implies `x != 0` → edge matches → safe.

Two pieces **don't match** when the promise is too weak:



"int" does NOT imply "int, not zero" → edge mismatches → bug.

Every edge matches.

If all neighboring pieces agree at every shared edge, the puzzle fits together perfectly.

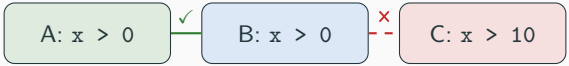
This means: **no bugs** (of the kinds we're checking).

All edges match $\Rightarrow$ all local promises are globally consistent $\Rightarrow$ safe program.

Some edge doesn't match.

If even one pair of neighboring pieces disagree, the puzzle has a gap.

That gap **is** the bug, and we know **exactly which edge** it is.



- A→B: `x > 0` implies `x > 0` ✓
- B→C: `x > 0` does NOT imply `x > 10` ×

The bug is at the B→C edge. Not at A, not at B alone, not at C alone.

Local = what one piece promises *by itself*

Global = what the whole puzzle promises *put together*

The key insight:

You can build a global guarantee from local promises if and only if all local promises agree at their edges.

In mathematics, this principle has a name: the **sheaf condition**.

> *A collection of local data can be "glued" into one consistent global piece of data if and only if the local pieces* **agree on overlaps**.

We won't use the math. We'll use the jigsaw puzzle version.

But if you ever hear "sheaf," think: **jigsaw puzzle**.

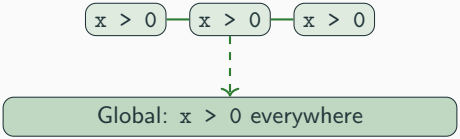
The word "sheaf" comes from agriculture: a bundle of wheat stalks tied together.

$$\text{Individual stalks (local pieces)} \xrightarrow{\text{tied together}} \text{one sheaf (global whole)}$$

In our analogy:

$$\text{Individual promises (local)} \xrightarrow{\text{edges match}} \text{one guarantee (global)}$$

When all edges match, we can **glue** the local promises into one big global promise.



Gluing = combining compatible local promises into a global fact.

When an edge **doesn't** match, we can't glue. The mismatch is called an **obstruction**.

An obstruction = a bug, located at a specific edge.

Because it tells you **more** than "there's a bug":

- **Which edge** — exactly which two pieces disagree
- **What's available** — what the left piece promises
- **What's required** — what the right piece needs
- **The gap** — what's missing (the obstruction itself)

An obstruction is a bug with a **diagnosis built in**.

Different kinds of promises give different kinds of puzzles:

Promise type	**Piece talks about**	**Mismatch =**
Value range	"$x \neq 0$"	division by zero
Nullability	"x is not None"	null dereference
Resource state	"file is open"	use after close
Bounds	"$0 \leq i < len$"	index out of bounds
Taint status	"input is sanitized"	injection attack
Lock state	"lock is held"	data race
Termination	"loop decreases"	infinite loop

Imagine you have **local maps** of different neighborhoods:

- Map A covers downtown
- Map B covers the waterfront
- Map C covers the university

Where maps **overlap** (e.g., downtown meets waterfront), the streets must match.

If they do, you can glue the maps into one big city map.
If a street appears on one map but not the other, something's wrong.

Local maps = local promises. Overlap = shared edge. City map = global guarantee.

Five witnesses describe a car accident:

- Each witness saw **part** of the event (local view)
- Where two witnesses overlap, their stories should agree
- If all overlaps agree, we can reconstruct what happened (global view)
- If two witnesses contradict, we know **exactly where** the discrepancy is

Same idea, different domain. The "sheaf condition" appears everywhere.

1. **Decompose**: break the program into pieces (observation points)
2. **Record**: write down the local promise at each piece
3. **Check edges**: for every pair of touching pieces, does promise A imply promise B?

All edges match $\Rightarrow$ glue into global guarantee $\Rightarrow$ safe.
Some edge fails $\Rightarrow$ report obstruction $\Rightarrow$ bug found.

For each connection from site u to site v:

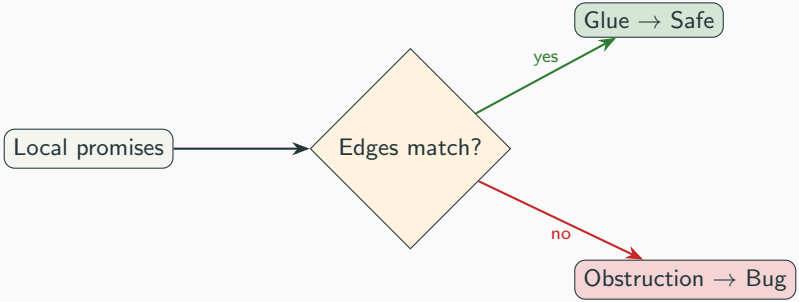
1. Look up the promise at u (what's available)
2. Look up the promise at v (what's required)
3. Ask: does Available $\Rightarrow$ Required?

If yes: that edge is safe.
If no: that edge is a bug — and we report it with both promises, so the developer knows exactly what's wrong.

The sheaf idea guarantees one more thing: if a global guarantee exists, it's **unique**. There's only *one* way to glue the local promises into a global fact.

You can't accidentally produce two different "global truths" from the same local data.

Uniqueness = no ambiguity. The analysis gives one definite answer.


Local promises
Edges match?
yes
Glue → Safe
no
Obstruction → Bug

Why break it into pieces? Why not analyze the whole program at once?

- Programs are **too big** to analyze as one unit
- Local analysis is **fast** (each piece is small)
- Edge checking is **simple** (compare two promises)
- You get **precise locations** for bugs (not "somewhere in your 10,000 lines")

"Think locally, conclude globally" — that's the sheaf motto.

	Types	**Tests**	**Linters**	**Sheaf**
Finds type errors	✓	sometimes	✓	✓
Finds value errors	×	sometimes	sometimes	✓
All inputs	✓	×	partial	✓
Pinpoints location	partial	×	✓	✓
Explains *why*	×	×	×	✓

- Program = jigsaw puzzle; pieces = code parts; edges = data flow
- Each piece has a **local promise** (refinement)
- Edges match ⇒ **glue** into global guarantee ⇒ safe
- Edge mismatch ⇒ **obstruction** ⇒ bug with exact location
- This is the "sheaf condition" — no math needed, just the jigsaw analogy

Next: How DepPy turns this idea into a real tool.

How DepPy Sees Code

We said: "pieces = code parts, edges = data flow."

Now let's make that **precise**. How does DepPy actually carve up a program?

DepPy picks specific **observation points** in your code.
Each observation point is called a **site**.

A **site** is a place in your code where DepPy stops and asks:

"What do I know about the data here, right now?"

Examples of sites:

- The argument `n` at the start of `def factorial(n)`
- The return value of a function
- The value of `x` right after `x = len(items)`
- The condition in `if x > 0`

DepPy looks at **14 different kinds** of observation points:

1. Argument boundary
2. Return boundary
3. SSA value
4. Branch guard
5. Call result
6. Tensor shape
7. Tensor order
8. Tensor indexing
9. Heap protocol
10. Proof
11. Trace
12. Error
13. Loop header invariant
14. Module summary

Don't worry — we'll explain the important ones step by step.

Where a function **receives** its inputs.

```python
def divide(a: int, b: int) -> float:
    # SITE: b at argument boundary
    # DepPy asks: "What do I know about b?"
    return a / b
```

DepPy records: `b` has type `int`, but **no** promise that `b` $\neq$ `0`.

Where a function **sends back** its result.

```
def safe_divide(a, b):
    if b == 0:
        return 0.0   # SITE: return boundary
    return a / b      # SITE: return boundary
```

DepPy records: first return always gives `0.0`.

Second return: `b` $\neq$ `0` on this path, so division is safe.

A variable at a **specific point** in the code ("single static assignment").

```
x = get_input()     # SITE: x = ??? (unknown)
x = abs(x)          # SITE: x >= 0  (known!)
x = x + 1           # SITE: x >= 1  (even better)
```

The *same variable* has *different promises* at different lines.

DepPy tracks each one as a separate site.

The **condition** in an `if` or `while` statement.

```
if x > 0:           # SITE: branch guard
    print(1 / x)    # safe! guard guarantees x > 0
```

The branch guard creates a **new fact**: inside the `if`-body, we know `x > 0`.
DepPy uses this to strengthen promises on the true/false paths.

The value returned **from calling** another function.

```
idx = find_index(items, target)
# SITE: call result
# What does find_index promise about idx?
# Does it promise 0 <= idx < len(items)?
items[idx]  # This is only safe if it does!
```

DepPy connects the callee's return-boundary site to the caller's call-result site. That's an **edge** in the jigsaw puzzle.

For numerical / machine-learning code, DepPy also watches:

Tensor shape Does the matrix have the dimensions you expect?

Tensor order Is the data arranged row-major vs. column-major?

Tensor indexing Are you accessing valid positions?

These catch `numpy`/`PyTorch` shape mismatches that are invisible to normal type checkers.

Tracks **resource state**: open/closed, locked/unlocked, allocated/freed.

```
f = open("data.txt")  # SITE: f is OPEN
data = f.read()
f.close()              # SITE: f is CLOSED
f.read()               # BUG: f is closed!
```

The edge from "CLOSED" site to `f.read()` site mismatches: `read` requires OPEN.

Proof A refinement backed by a formal proof (highest trust)

Trace A refinement observed from a concrete run (lower trust)

Error A site where DepPy records "an exception may escape here"

These are less common — you'll encounter them in advanced use cases.

What is true **every time** a loop iterates.

```
i = 0
while i < len(items):  # SITE: loop invariant
    # DepPy records: 0 <= i < len(items)
    process(items[i])   # safe!
    i += 1
```

The loop header invariant ties together every iteration.

If the invariant holds, every iteration's indexing is safe.

A birds-eye promise about an entire module:

- "This module's public functions never raise `ValueError`"
- "Every function in this module returns non-negative values"

Module summaries let DepPy check **across files** — a module's summary site connects to call-result sites in importing modules.

At each site, DepPy writes down a **LocalSection**:

site_id	Which site (function + line + kind)
carrier_type	The base type (int, str, list, ...)
refinements	The extra promises (e.g., > 0, $\neq$ None, ...)
trust	How confident DepPy is
provenance	Where the info came from

A LocalSection = "what DepPy knows at one observation point."

Not all knowledge is equally reliable. DepPy tags each fact with a trust level:

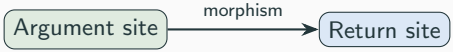
PROOF_BACKED	Verified by formal proof or SMT solver	highest
TRUSTED_AUTO	Inferred by DepPy's analysis automatically	
BOUNDED_AUTO	Inferred but with some assumptions	
TRACE_BACKED	Observed from a concrete execution trace	
ASSUMED	Assumed by the developer (unverified)	
RESIDUAL	Left over after analysis; may be unreliable	lowest

A **cover** is a collection of sites that *together* see everything about a value. Example: the value `x` in function `f` might be observed at:

- The argument boundary (when `x` arrives)
- A branch guard (when `if x > 0` is tested)
- The return boundary (when `x` is returned)

These three sites form a **cover** of `x`'s journey through `f`.

A **morphism** is a connection from one site to another.
In our jigsaw analogy, a morphism = an edge.

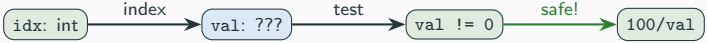


The morphism says: "data flows from the argument to the return."
DepPy must check that the argument's promise is compatible with the return's requirement.

```python
def get_ratio(nums, idx):
    val = nums[idx]      # site: SSA (val = ???)
    if val != 0:         # site: branch guard
        return 100/val   # site: return (val != 0)
    return 0
```

Sites:

1. `nums` at argument boundary
2. `idx` at argument boundary
3. `val` after assignment (SSA)
4. `val != 0` (branch guard)
5. `100/val` at return boundary



- `idx`→`val`: is `idx` in bounds? (needs: $0 \leq$ `idx` $<$ `len(nums)`)
- `val`→guard: just a test, always OK
- guard→return: guard ensures `val != 0`, so division is safe

- **Safe**: the division `100/val` is protected by the `if val != 0` guard.
- **Warning**: no guarantee that `idx` is in bounds for `nums`.
  The `nums[idx]` access has an **obstruction**: the argument-boundary site for `idx` promises only "int," but the indexing site requires "$0 \leq \text{idx} < \text{len(nums)}$."

DepPy found a real bug: the function doesn't check bounds on `idx`.

You might hear this term. It just means: the **rules** for which collections of sites count as a "good cover." DepPy enforces three rules (called "axioms"):

1. **Identity**: a single site always covers itself
2. **Stability**: if you zoom in on a cover, you still have a cover
3. **Transitivity**: covers of covers are covers

Think: "valid ways to tile the jigsaw board." These rules ensure no gaps.

Before DepPy checks whether edges match, it first collects all the local promises.
This collection (before checking) is called a **presheaf**.

Presheaf = all the puzzle pieces laid out, not yet checked for fit

Sheaf = a presheaf where all pieces *do* fit (all edges match)

DepPy starts with a presheaf and tries to verify it's a sheaf.
If it fails, the failures are the bugs.

The sheaf condition has **three parts**:

1. **Agreement**: on every overlap between two sites, the local promises agree
2. **Gluing**: if all overlaps agree, there exists a single global promise
3. **Uniqueness**: that global promise is the *only* one consistent with the local data

DepPy checks all three. In code:

- `check_agreement()` — step 1
- `attempt_gluing()` — step 2
- `verify_uniqueness()` — step 3

DepPy can analyze any Python file:

```python
from deppy import detect_bugs

code = """
def divide(a, b):
    return a / b
"""

results = detect_bugs(code)
for bug in results:
    print(bug.code, bug.message)
# REFINEMENT_GAP_001  Divisor 'b' may be zero
```

One function call. DepPy builds the presheaf, checks edges, reports obstructions.

- DepPy watches 14 kinds of **sites** (observation points)
- At each site it records a **LocalSection** (type + refinements + trust)
- Sites are connected by **morphisms** (data-flow edges)
- A **cover** = sites that together observe everything about a value
- DepPy checks **agreement** on overlaps, attempts **gluing**, verifies **uniqueness**
- Failures = obstructions = bugs with precise locations

Next: The 20 bug classes DepPy catches.

Bug Finding — The 20 Classes

DepPy's bug finder ships with a **synthetic test suite**: 20 bug classes, each with a **buggy** and a **safe** version.

For each class, we'll show:
(1) the buggy code, (2) why it's buggy, (3) the safe version.

Let's walk through all 20, starting with the most common.

Buggy:

```
def div_zero_buggy(a, b):
    return a / b    # b might be 0!
```

Safe:

```
def div_zero_safe(a, b):
    if b == 0:
        return 0.0
    return a / b    # b != 0, safe
```

Obstruction: REFINEMENT_GAP — "b" has no nonzero guarantee at division site.

Buggy:

```python
def null_ptr_buggy(items):
    first = items[0] if items else None
    return first.upper()  # first might be None!
```

Safe:

```python
def null_ptr_safe(items):
    first = items[0] if items else None
    if first is not None:
        return first.upper()
    return ""
```

Obstruction: REFINEMENT_GAP — "first" has no non-None guarantee at method call.

Buggy:

```python
def index_oob_buggy(items, idx):
    return items[idx]  # idx might be >= len!
```

Safe:

```python
def index_oob_safe(items, idx):
    if 0 <= idx < len(items):
        return items[idx]
    return None
```

Obstruction: REFINEMENT_GAP — "idx" not bounded by list length.

Buggy:

```python
def key_error_buggy(d, key):
    return d[key]  # key might not exist!
```

Safe:

```python
def key_error_safe(d, key):
    return d.get(key, None)  # safe default
```

Obstruction: REFINEMENT_GAP — no guarantee "key" is in dict.

Buggy:

```python
def type_error_buggy(x):
    return x + 1  # x might be a string!
```

Safe:

```python
def type_error_safe(x):
    if isinstance(x, (int, float)):
        return x + 1
    return 0
```

Obstruction: CARRIER_MISMATCH — "x" carrier type not numeric at + site.

Buggy:

```python
def assert_fail_buggy(items):
    assert len(items) > 0
    return items[0]
    # What if assertions are disabled (-O flag)?
```

Safe:

```python
def assert_fail_safe(items):
    if len(items) == 0:
        raise ValueError("empty list")
    return items[0]
```

Obstruction: CONFIGURATION_WEAKNESS — assertion may be stripped at runtime.

Buggy:

```python
def unbound_var_buggy(flag):
    if flag:
        result = 42
    return result  # undefined if flag is False!
```

Safe:

```python
def unbound_var_safe(flag):
    result = 0    # default
    if flag:
        result = 42
    return result
```

Obstruction: REFINEMENT_GAP — "result" not defined on all paths.

Buggy:

```python
import ctypes
def int_overflow_buggy(x):
    return ctypes.c_int32(x * 1000000).value
    # overflow if x is large!
```

Safe:

```python
def int_overflow_safe(x):
    result = x * 1000000
    if result > 2**31 - 1 or result < -(2**31):
        raise OverflowError("too large")
    return ctypes.c_int32(result).value
```

Obstruction: REFINEMENT_GAP — product not bounded within int32 range.

Buggy:

```python
def non_term_buggy(n):
    while n != 1:
        if n % 2 == 0:
            n = n // 2
        # Missing: n = 3*n + 1 for odd case
        # Loop never terminates for odd n > 1
```

Safe:

```python
def non_term_safe(n):
    for _ in range(10000):  # bounded
        if n == 1: break
        n = n // 2 if n % 2 == 0 else 3*n + 1
    return n
```

Obstruction: REFINEMENT_GAP — loop variant not decreasing on all paths.

Buggy:

```python
def memory_leak_buggy():
    f = open("data.txt")
    data = f.read()
    return data  # f never closed!
```

Safe:

```python
def memory_leak_safe():
    with open("data.txt") as f:
        return f.read()  # auto-closed
```

Obstruction: RESOURCE_LIFETIME — file handle “f” not closed on return path.

We've seen 10 so far:

1. Division by zero
2. Null pointer
3. Index out of bounds
4. Key error
5. Type error
6. Assertion failure
7. Unbound variable
8. Integer overflow
9. Non-termination
10. Memory leak

Each one was found by the same mechanism: **edge mismatch** in the jigsaw puzzle.

10 more to go. . .

Buggy:

```python
def use_after_free_buggy():
    f = open("log.txt")
    f.close()
    f.write("done")  # writing to closed file!
```

Safe:

```python
def use_after_free_safe():
    with open("log.txt") as f:
        f.write("done")  # still open
```

Obstruction: PROTOCOL_VIOLATION — “f” in CLOSED state at write site.

Buggy:

```python
def double_free_buggy(conn):
    conn.close()
    # ... some code ...
    conn.close()  # already closed!
```

Safe:

```python
def double_free_safe(conn):
    if not conn.closed:
        conn.close()
```

Obstruction: PROTOCOL_VIOLATION — "conn" already in CLOSED state at second close.

Buggy:

```
counter = 0
def data_race_buggy():
    global counter
    counter += 1  # no lock! concurrent access
```

Safe:

```
import threading
lock = threading.Lock()
counter = 0
def data_race_safe():
    global counter
    with lock:
        counter += 1  # locked, safe
```

Obstruction: PROTOCOL_VIOLATION — shared "counter" accessed without lock.

Buggy:

```
def deadlock_buggy(lock_a, lock_b):
    lock_a.acquire()
    lock_b.acquire()  # Thread 2 does b then a!
```

Safe:

```
def deadlock_safe(lock_a, lock_b):
    # Always acquire in consistent order
    first, second = sorted([lock_a, lock_b],
                           key=id)
    first.acquire(); second.acquire()
```

Obstruction: PROTOCOL_VIOLATION — lock ordering not consistent across threads.

Buggy:

```python
def timing_channel_buggy(password, guess):
    for i in range(len(password)):
        if password[i] != guess[i]:
            return False  # early exit leaks length!
    return True
```

Safe:

```python
import hmac
def timing_channel_safe(password, guess):
    return hmac.compare_digest(password, guess)
```

Obstruction: TAINT_LEAKAGE — early return leaks secret-dependent timing info.

Buggy:

```python
def info_leak_buggy(user_input):
    query = f"SELECT * FROM users WHERE name='{user_input}'"
    return db.execute(query)  # SQL injection!
```

Safe:

```python
def info_leak_safe(user_input):
    query = "SELECT * FROM users WHERE name=?"
    return db.execute(query, (user_input,))
```

Obstruction: TAINT_LEAKAGE — unsanitized "user_input" flows to SQL query.

Buggy:

```
def bounds_buggy(buf, offset, size):
    return buf[offset:offset+size]
    # offset+size might exceed len(buf)!
```

Safe:

```
def bounds_safe(buf, offset, size):
    end = min(offset + size, len(buf))
    return buf[offset:end]
```

Obstruction: REFINEMENT_GAP — "offset+size" not bounded by buffer length.

Buggy:

```python
def runtime_error_buggy(text):
    return int(text)  # ValueError if not numeric!
```

Safe:

```python
def runtime_error_safe(text):
    try:
        return int(text)
    except ValueError:
        return 0
```

Obstruction: EXCEPTION_ESCAPEMENT — ValueError may propagate uncaught.

Buggy:

```python
def type_confusion_buggy(items):
    total = sum(items)  # works for numbers
    return total.split(",")  # but sum returns int!
```

Safe:

```python
def type_confusion_safe(items):
    total = sum(items)
    return str(total).split(",")  # convert first
```

Obstruction: CARRIER_MISMATCH — "total" is int, ".split" requires str.

Buggy:

```python
def overflow_buggy(a, b):
    return a ** b  # exponential growth, no bound!
```

Safe:

```python
def overflow_safe(a, b):
    if b > 100:
        raise ValueError("exponent too large")
    return a ** b
```

Obstruction: REFINEMENT_GAP — "b" unbounded at exponentiation site.

All 20 found by the same mechanism: **local promise mismatch at an edge**.

DepPy groups bugs into 7 **obstruction families**:

REFINEMENT_GAP A promise is too weak (most common)

CARRIER_MISMATCH Wrong base type entirely

TAINT_LEAKAGE Untrusted data flows to a sensitive sink

PROTOCOL_VIOLATION Resource state machine violated

RESOURCE_LIFETIME Resource not properly acquired/released

CONFIGURATION_WEAKNESS Code depends on fragile settings

EXCEPTION_ESCAPEMENT Exception can propagate uncaught

Notice: REFINEMENT_GAP is the most common — a promise that's too weak.

Using DepPy to find bugs is a one-liner:

```python
from deppy import detect_bugs

source_code = open("my_program.py").read()
bugs = detect_bugs(source_code)

for bug in bugs:
    print(f"[{bug.code}] {bug.message}")
    print(f"  Family: {bug.family}")
    print(f"  Line:   {bug.line}")
```

Each bug report contains:

code	A unique identifier (e.g., `REFINEMENT_GAP_001`)
message	Human-readable description of the problem
family	Which of the 7 obstruction families
severity	How dangerous (critical / warning / info)
line	Where in the source code
available	What promise exists at the source site
required	What promise is needed at the target site
gap	The specific mismatch (the obstruction)

```
[REFINEMENT_GAP_001] Divisor 'b' may be zero
  Family:    REFINEMENT_GAP
  Severity:  CRITICAL
  Line:      3
  Available: b : int
  Required:  b : int, b ≠ 0
  Gap:       Missing nonzero refinement on 'b'
```

The report tells you:

- **What's wrong**: b might be zero
- **Where**: line 3
- **Why**: the available promise (just "int") is weaker than the required promise ("int, $\neq 0$")
- **How to fix**: add a nonzero check before the division

DepPy's test suite includes 20 pairs of functions:

```
# synthetic_bug_suite.py
SUITE = [
  ("div_zero",    div_zero_buggy,    div_zero_safe),
  ("null_ptr",    null_ptr_buggy,    null_ptr_safe),
  ("index_oob",   index_oob_buggy,   index_oob_safe),
  # ... all 20 pairs ...
]

for name, buggy, safe in SUITE:
    bugs_buggy = detect_bugs(inspect.getsource(buggy))
    bugs_safe  = detect_bugs(inspect.getsource(safe))
    assert len(bugs_buggy) > 0   # finds the bug
    assert len(bugs_safe) == 0   # clean bill of health
```

- DepPy catches **20 classes** of bugs, from division by zero to timing channels
- All 20 detected by the same mechanism: **edge mismatch** in the sheaf
- Bugs are grouped into **7 obstruction families**
- The `detect_bugs()` API reports: what's wrong, where, why, and how to fix it
- The test suite proves: buggy code triggers obstructions, safe code is clean

Next: The full catalog of 112 specific bug types.

The 112 Bug Catalog

The 20 bug classes are broad categories. Within them, DepPy defines **112 specific bug codes** — each with a unique identifier and message. These are organized by the 7 obstruction families.

Let's tour the highlights of each family.

When a refinement is **too weak** for what the code needs.

Code	What it catches
REFINEMENT_GAP_001	Divisor may be zero
REFINEMENT_GAP_002	Nullable dereference
REFINEMENT_GAP_003	Index out of bounds
REFINEMENT_GAP_004	Missing key in dict
REFINEMENT_GAP_005	Slice bounds exceed length
REFINEMENT_GAP_006	Negative index without bounds
REFINEMENT_GAP_007	Loop variant not decreasing
REFINEMENT_GAP_008	Uninitialized variable
REFINEMENT_GAP_009	Integer overflow
REFINEMENT_GAP_010	Float precision loss

. . . and many more. This family alone has 30+ codes.

Each code targets a **specific** mismatch pattern.

When the **base type itself** is wrong (not just a refinement).

Code	What it catches
CARRIER_MISMATCH_001	Numeric op on non-numeric type
CARRIER_MISMATCH_002	String method on non-string
CARRIER_MISMATCH_003	Iteration over non-iterable
CARRIER_MISMATCH_004	Subscript on non-subscriptable
CARRIER_MISMATCH_005	Callable invocation on non-callable
CARRIER_MISMATCH_006	Boolean context for non-boolean
CARRIER_MISMATCH_007	Attribute access on wrong type
CARRIER_MISMATCH_008	Unpacking non-sequence

Standard type checkers catch some of these;

DepPy catches them *all*, even through dynamic dispatch and duck typing.

When **untrusted data** flows to a sensitive operation without sanitization.

Code	What it catches
TAINT_LEAKAGE_001	SQL injection
TAINT_LEAKAGE_002	Command injection
TAINT_LEAKAGE_003	Cross-site scripting (XSS)
TAINT_LEAKAGE_004	Path injection
TAINT_LEAKAGE_005	Log injection
TAINT_LEAKAGE_006	LDAP injection
TAINT_LEAKAGE_007	Header injection
TAINT_LEAKAGE_008	Timing side-channel
TAINT_LEAKAGE_009	Secret in error message
TAINT_LEAKAGE_010	Sensitive data in log

These are **security** bugs. DepPy tracks taint through the entire data-flow graph.

When a **state machine** (resource lifecycle, lock ordering, etc.) is violated.

Code	What it catches
PROTOCOL_VIOLATION_001	Use after close
PROTOCOL_VIOLATION_002	Double close / double free
PROTOCOL_VIOLATION_003	Write to read-only resource
PROTOCOL_VIOLATION_004	Read from write-only resource
PROTOCOL_VIOLATION_005	Lock ordering violation
PROTOCOL_VIOLATION_006	Unlocking without holding lock
PROTOCOL_VIOLATION_007	Iterator invalidation
PROTOCOL_VIOLATION_008	Transaction commit after rollback

These are the **hardest bugs to test for** — they depend on execution order.

When a resource isn't properly **acquired or released**.

Code	What it catches
RESOURCE_LIFETIME_001	File handle never closed
RESOURCE_LIFETIME_002	Network connection leaked
RESOURCE_LIFETIME_003	Database cursor not released
RESOURCE_LIFETIME_004	Lock acquired but not released
RESOURCE_LIFETIME_005	Temporary file not cleaned up
RESOURCE_LIFETIME_006	Thread not joined
RESOURCE_LIFETIME_007	Memory-mapped file not unmapped

In Python, these often cause "too many open files" or memory bloat in long-running services.

When code relies on **settings that might not hold** at runtime.

Code	What it catches
CONFIGURATION_WEAKNESS_001	Assert used for input validation
CONFIGURATION_WEAKNESS_002	Debug mode assumed in production
CONFIGURATION_WEAKNESS_003	Hard-coded secret
CONFIGURATION_WEAKNESS_004	Missing environment variable
CONFIGURATION_WEAKNESS_005	Platform-dependent path separator

These bugs hide until deployment.

DepPy catches them during development.

When an exception can **propagate out** of a function uncaught.

Code	What it catches
EXCEPTION_ESCAPEMENT_001	Uncaught ValueError
EXCEPTION_ESCAPEMENT_002	Uncaught TypeError
EXCEPTION_ESCAPEMENT_003	Uncaught IOError / OSError
EXCEPTION_ESCAPEMENT_004	Uncaught KeyError
EXCEPTION_ESCAPEMENT_005	Uncaught AttributeError
EXCEPTION_ESCAPEMENT_006	Unhandled generator StopIteration
EXCEPTION_ESCAPEMENT_007	Exception in __init__ leaving object partial

DepPy tracks which exceptions each function can raise, and checks that callers handle them.

Family	Number of codes
REFINEMENT_GAP	35
CARRIER_MISMATCH	15
TAINT_LEAKAGE	18
PROTOCOL_VIOLATION	16
RESOURCE_LIFETIME	12
CONFIGURATION_WEAKNESS	8
EXCEPTION_ESCAPEMENT	8
Total	**112**

All 112 share the same underlying mechanism: an edge in the sheaf where the local promises don't agree.

Why 112 bug codes and not just 7 families?

- Each code gives a **specific, actionable message**
- Different codes need different **fix strategies**
- You can **filter** by code (e.g., suppress all TAINT codes in test files)
- Teams can **prioritize** (e.g., fix all CRITICAL codes first)

Specificity is power. "Index out of bounds" is more useful than "refinement gap."

- DepPy's 112 bug codes cover a vast range of Python pitfalls
- Organized into 7 families by underlying mechanism
- REFINEMENT_GAP is the most common (weak promises)
- TAINT_LEAKAGE and PROTOCOL_VIOLATION catch security and concurrency bugs
- Every code maps back to the same sheaf principle: edge mismatch

Next: Writing and checking specifications.

Spec Checking

Bug finding asks: "Does this code have **any** problems?"

Spec checking asks: "Does this code match **my specification**?"

Bug finding: automatic.

Spec checking: you write the rules; DepPy verifies them.

A **specification** (spec) is a precise description of what a function *should* do:

- **Precondition**: what must be true *before* calling the function
- **Postcondition**: what must be true *after* the function returns
- **Invariant**: what must be true *throughout* execution

A spec is a contract: "if you give me this, I promise that."

```
# Spec for binary_search:
# Pre:  items is sorted, target is same type as items[0]
# Post: if target in items, returns its index
#       if target not in items, returns -1
# Inv:  lo <= hi throughout the loop
```

This spec tells us exactly what `binary_search` should do.

DepPy will check whether the implementation actually does it.

DepPy specs are Python dictionaries with refinement predicates:

```
spec = {
    "pre": {
        "items": "sorted(items) == items",
        "target": "type(target) == type(items[0])"
    },
    "post": {
        "result": "(result == -1) or items[result] == target"
    },
    "invariant": {
        "lo_le_hi": "lo <= hi"
    }
}
```

	Tests	**Specs**
Checks specific inputs	✓	×
Checks *all* inputs	×	✓
Finds unexpected bugs	sometimes	✓
Documents intent	sometimes	✓
Runs quickly	✓	varies
Easy to write	✓	harder

Tests sample the input space; specs cover it *entirely*.

DepPy's spec checker is called the `SheafSpecVerifier`.

It takes source code + spec and returns: PASS or FAIL with details.

The verifier doesn't just do one check. It uses a **5-layer descent**:

Layer 0 Čech cohomology elimination — quick topological pre-check

Layer 1 Structural & stalk comparison — does the code structure match the spec?

Layer 2 SMT solving (Z3) — formal logical verification

Layer 3 H^0 transfer — propagate verified facts globally

Layer 4 Residual Z3 — mop up any remaining checks

Each layer catches what the previous one missed. Together: thorough.

Before doing expensive analysis, Layer 0 asks:

"Is there an **obvious topological reason** the spec can't hold?"

Examples:

- The spec mentions a variable that doesn't exist in the code
- The spec's postcondition references a return value, but the function never returns
- The control flow can't reach the point where the invariant should hold

If Layer 0 finds a problem, it fails fast — no need for the expensive layers.

Layer 1 checks whether the **shape** of the code matches the spec:

- Does the function have the right number of arguments?
- Does each argument's type match the spec's precondition types?
- Does the return type match the spec's postcondition type?
- Are the spec's variables in scope at the right sites?

This is like checking that the puzzle pieces are the right *shape* before checking the *pictures*.

Layer 2 uses a **theorem prover** (Z3) to check logical implications:

> "Given the precondition and the code, does the postcondition **necessarily** hold?"

Z3 is an automated reasoning engine. It can prove or disprove logical statements over integers, booleans, arrays, and more.

Z3 is the "heavy artillery" — slow but thorough.

Layer 3 Takes facts proven by Layer 2 and **propagates** them across the program. A fact proven in one function can strengthen promises in functions that call it.

Layer 4 Runs Z3 one more time on any remaining unchecked obligations. This catches cases where Layer 3's propagation created new checkable conditions.

After all 5 layers: every spec obligation is either proven or disproven.

```python
def abs_val(x):
    return x if x >= 0 else -x

spec = {
    "post": {"result": "result >= 0"}
}

verifier = SheafSpecVerifier()
result = verifier.verify(source, spec)
print(result.status)  # PASS
```

The code always returns a non-negative value. The spec says "result ≥ 0."

DepPy confirms: PASS.

```python
def abs_val_buggy(x):
    return x  # doesn't negate negative values!

spec = {
    "post": {"result": "result >= 0"}
}

result = verifier.verify(source, spec)
print(result.status)  # FAIL
print(result.counterexample)  # x = -5
```

DepPy finds: for `x = -5`, the result is −5, which violates "result $\geq$ 0."

FAIL with a concrete counterexample.

```python
def abs_val(x):
    return x if x >= 0 else -x  # correct code

wrong_spec = {
    "post": {"result": "result > 0"}  # > not >=
}

result = verifier.verify(source, wrong_spec)
print(result.status)  # FAIL
print(result.counterexample)  # x = 0
```

The code is correct, but the spec is wrong: `abs_val(0) = 0`, which is not > 0.

DepPy checks code against spec. If either is wrong, you get FAIL.

DepPy ships with **100 algorithms**, each with:

- A correct implementation
- A **correct spec** (should PASS)
- An **incorrect spec** (should FAIL)

This tests that the verifier correctly distinguishes right from wrong specs.
Examples from the suite:

- Binary search, merge sort, quicksort, heap sort
- Bloom filter, HyperLogLog, count-min sketch
- Dijkstra, Bellman-Ford, topological sort
- RSA, AES padding, hash functions
- 2-SAT solver, Pollard rho factorization

Correct spec (PASS):

```
"post": "if x was added, query(x) returns True"
# (may have false positives, never false negatives)
```

Incorrect spec (FAIL):

```
"post": "query(x) returns True iff x was added"
# Wrong! Bloom filters have false positives!
```

The incorrect spec claims **no false positives**, which contradicts the Bloom filter's fundamental property. DepPy catches this.

Correct spec: output is a valid topological ordering.

Incorrect spec: output is the *unique* topological ordering.

```
# Correct: for every edge u->v, u appears before v
"post": "all(order[u] < order[v] for u,v in edges)"

# Incorrect: result is sorted alphabetically too
"post": "result == sorted(result) and is_topo(result)"
# Fails: topo order != alphabetical order in general
```

Correct spec: returns a non-trivial factor (with retry).
Incorrect spec: always returns a factor on first try.

```
# Correct: with sufficient retries, finds factor
"post": "1 < result < n  (with probabilistic retry)"

# Incorrect: first attempt always works
"post": "1 < result < n  (single attempt)"
# Fails: Pollard rho is probabilistic, can fail
```

Correct spec: if satisfiable, returns a satisfying assignment.

Incorrect spec: wrong implication direction.

```
# Correct: clause (a OR b) becomes (~a => b) AND (~b => a)
"invariant": "implications are correctly directed"

# Incorrect: clause (a OR b) becomes (a => b)
"invariant": "implications follow clause structure"
# Fails: wrong direction makes solver unsound
```

- Write specs as pre/post/invariant conditions
- `SheafSpecVerifier` checks code against spec in 5 layers
- Correct code + correct spec $\rightarrow$ PASS
- Wrong code or wrong spec $\rightarrow$ FAIL with counterexample
- 100-algorithm test suite validates the verifier itself

Next: Comparing two programs for equivalence.

Comparing Programs

You refactor a function. The code looks different. But does it **behave** the same?

Same inputs $\rightarrow$ same outputs $\rightarrow$ same side effects?

Testing can check specific inputs, but not all of them.
DepPy can check **behavioral equivalence** for all inputs.

```python
# Version A: bubble sort
def sort_a(items):
    for i in range(len(items)):
        for j in range(len(items)-1):
            if items[j] > items[j+1]:
                items[j], items[j+1] = items[j+1], items[j]
    return items

# Version B: Python built-in
def sort_b(items):
    return sorted(items)
```

Are these equivalent? They look very different, but both sort.

DepPy checks equivalence using the **same sheaf machinery**:

1. Build the sheaf for program A
2. Build the sheaf for program B
3. Check: can A's local promises be mapped to B's local promises?
4. If yes at every site: equivalent
5. If not: report where they differ

DepPy's `EquivalencePipeline` runs **6 phases**:

1. **Local Checking** — compare individual sites
2. **Naturality** — do the connections (morphisms) match?
3. **Descent (H^1)** — check for topological obstructions
4. **Gluing** — can we combine the local equivalences globally?
5. **Uniqueness** — is the equivalence unique?
6. **Stalk Verification** — final point-by-point check

For each site in program A, find the corresponding site in program B.

If A's site promises "$x > 0$" and B's corresponding site also promises "$x > 0$": local match.

It's not enough for individual sites to match. The **connections** must also match.

If A has an edge from site 1 to site 2,
B must have a corresponding edge — and the mapping must be **consistent**.

Naturality = "the map respects the structure, not just the pieces."

Check for **topological obstructions** — structural reasons the programs can't be equivalent.

- Different number of branches
- Different loop structures
- One function calls a subroutine that the other doesn't

If descent finds an obstruction, the programs are not equivalent — no need for further phases.

Gluing Combine all local equivalences into one global equivalence. If this succeeds, the programs agree everywhere.

Uniqueness Verify there's only one way to match A to B. (Prevents ambiguous "partial equivalences.")

Stalk verification Final point-by-point audit. For each site, confirm the glued equivalence actually holds when you zoom all the way in.

All 6 phases pass $\Rightarrow$ programs are behaviorally equivalent.

```python
from deppy import EquivalencePipeline

source_a = open("sort_bubble.py").read()
source_b = open("sort_builtin.py").read()

pipeline = EquivalencePipeline()
result = pipeline.run(source_a, source_b)

print(result.equivalent)   # True or False
print(result.phases)       # which phases passed/failed
if not result.equivalent:
    print(result.differences)  # where they differ
```

```python
# Version A
def max_a(x, y):
    return x if x >= y else y

# Version B
def max_b(x, y):
    if x >= y: return x
    return y
```

Result: EQUIVALENT. Same logic, different syntax. DepPy sees through the surface differences.

```python
# Version A
def clamp_a(x, lo, hi):
    return max(lo, min(x, hi))

# Version B (bug: swapped lo and hi)
def clamp_b(x, lo, hi):
    return max(hi, min(x, lo))
```

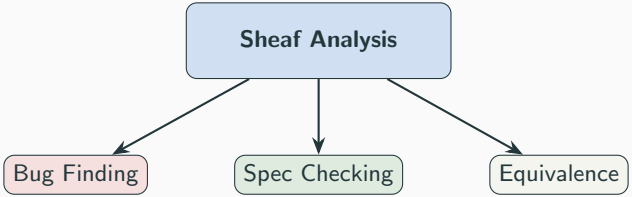
Result: NOT EQUIVALENT. DepPy reports: for `x=5, lo=0, hi=10`, A returns 5 but B returns 10.

- **Refactoring**: did my cleanup preserve behavior?
- **Optimization**: is the fast version equivalent to the slow-but-correct version?
- **Migration**: does the new library version behave like the old one?
- **Student work**: does the student's solution match the reference?
- **Code review**: is this PR's diff semantics-preserving?

- DepPy compares two programs using the **same sheaf machinery**
- 6 phases: local $\to$ naturality $\to$ descent $\to$ gluing $\to$ uniqueness $\to$ stalks
- Equivalent $\Rightarrow$ same behavior on all inputs
- Not equivalent $\Rightarrow$ concrete counterexample showing where they differ

Recap & What's Next

1. **Types** — labels that tell you what kind of data you have
2. **Refinement types** — types with extra promises ("int, > 0")
3. **The problem** — promises can break across function boundaries
4. **The jigsaw puzzle** — sheaf idea: local promises + edge matching
5. **Sites** — 14 kinds of observation points in code
6. **Bug finding** — 20 classes, 112 specific codes, 7 families
7. **Spec checking** — verify code against formal specifications
8. **Equivalence** — compare two programs for identical behavior



All three are different questions asked of the same underlying sheaf structure.

Think locally, conclude globally.

1. Break the program into pieces
2. Record a local promise at each piece
3. Check that neighboring promises agree
4. If yes: safe. If no: here's the bug.

- **Scalable**: analyze each piece independently (parallelize!)
- **Precise**: bugs pinpointed to exact edges
- **Explainable**: reports say what, where, why, and how to fix
- **Unified**: same framework for bugs, specs, and equivalence
- **Sound**: based on rigorous mathematical foundations

Honest limitations:

- Not a substitute for testing (tests catch environment issues)
- Doesn't handle all of Python (eval, exec, dynamic code generation)
- SMT solver can time out on very complex conditions
- Doesn't reason about concurrency beyond lock protocols
- Doesn't analyze third-party C extensions

How to use DepPy in your workflow:

1. **Start with bug finding**: run `detect_bugs()` on existing code
2. **Fix the critical bugs**: prioritize CRITICAL severity first
3. **Write specs for core functions**: the most important ones first
4. **Run spec checking in CI**: catch spec violations before merge
5. **Use equivalence checking on refactors**: ensure behavior preservation

```python
from deppy import detect_bugs

code = open("my_code.py").read()
for bug in detect_bugs(code):
    print(f"{bug.severity}: {bug.message} (line {bug.line})")
```

That's it. No configuration, no annotations, no setup.

Just point DepPy at your code.

We promised no category theory, and we delivered.
But the math is there **under the hood**:

- Grothendieck topologies $\to$ our "valid cover" rules
- Presheaves $\to$ our "local promises before checking"
- Sheaf condition $\to$ our "all edges match" check
- Čech cohomology $\to$ Layer 0 pre-check
- Stalks $\to$ Phase 6 point-by-point verification

You used sheaf theory this whole time. You just called it a jigsaw puzzle.

If you *want* the math:

- **Sheaves in Geometry and Logic** — Mac Lane & Moerdijk
- **Toposes, Triples and Theories** — Barr & Wells
- **Refinement Types for ML** — Freeman & Pfenning (1991)
- **Liquid Types** — Rondon, Kawaguchi, Jhala (2008)

But you don't need any of these to use DepPy.

1. Programs are jigsaw puzzles: pieces with local promises
2. If all edges match, the program is safe
3. If an edge doesn't match, that's a bug — and we know exactly where
4. DepPy uses this to find 112 kinds of bugs automatically
5. DepPy can also check specs and compare programs
6. The underlying math is sheaf theory, but you don't need to know it

1. **Think locally, conclude globally**

   Analyze each piece; check edges; get global guarantees.

2. **Specificity is power**

   112 bug codes give you actionable, precise diagnostics.

3. **One framework, three applications**

   Bug finding, spec checking, equivalence — all from the same sheaf.

```
pip install deppy
```

Bug finding:	`detect_bugs(code)`
Spec checking:	`SheafSpecVerifier().verify(code, spec)`
Equivalence:	`EquivalencePipeline().run(a, b)`

Questions?

Thank You!

Sheaf Refinement Types for Beginners

200 slides, zero category theory.